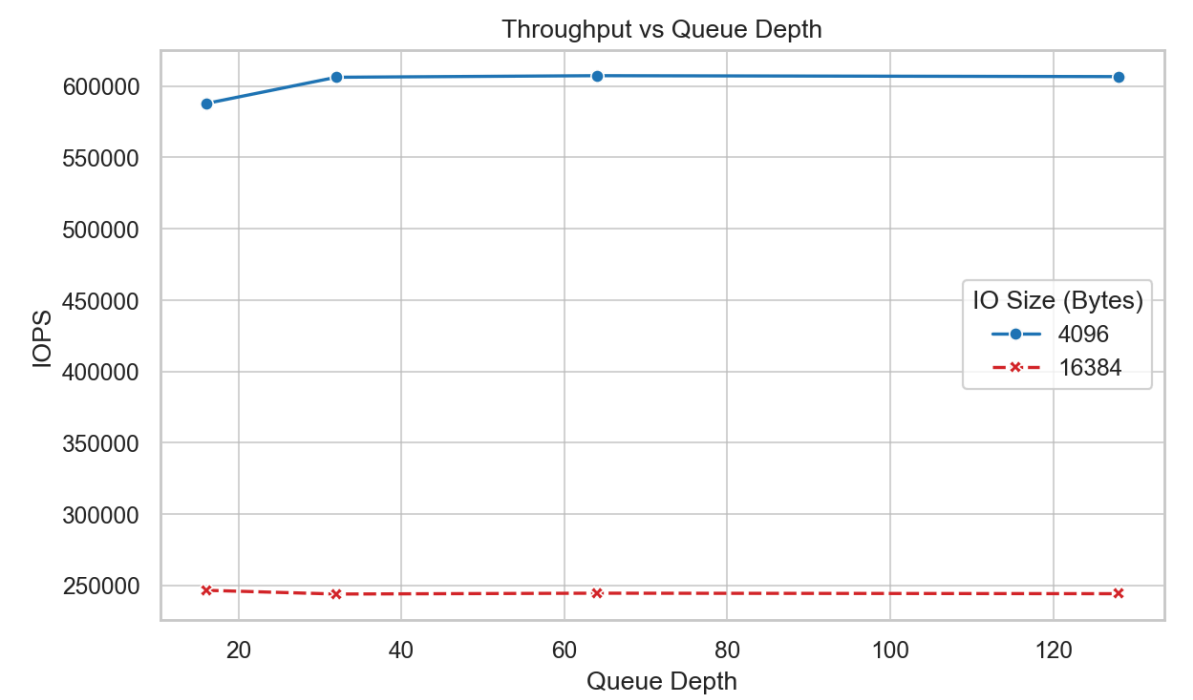


Phase 1 Comprehensive Datasheet

Generated from `phase1_results.csv`. This report covers Performance, CPU Efficiency, Memory Subsystem, and Signaling.

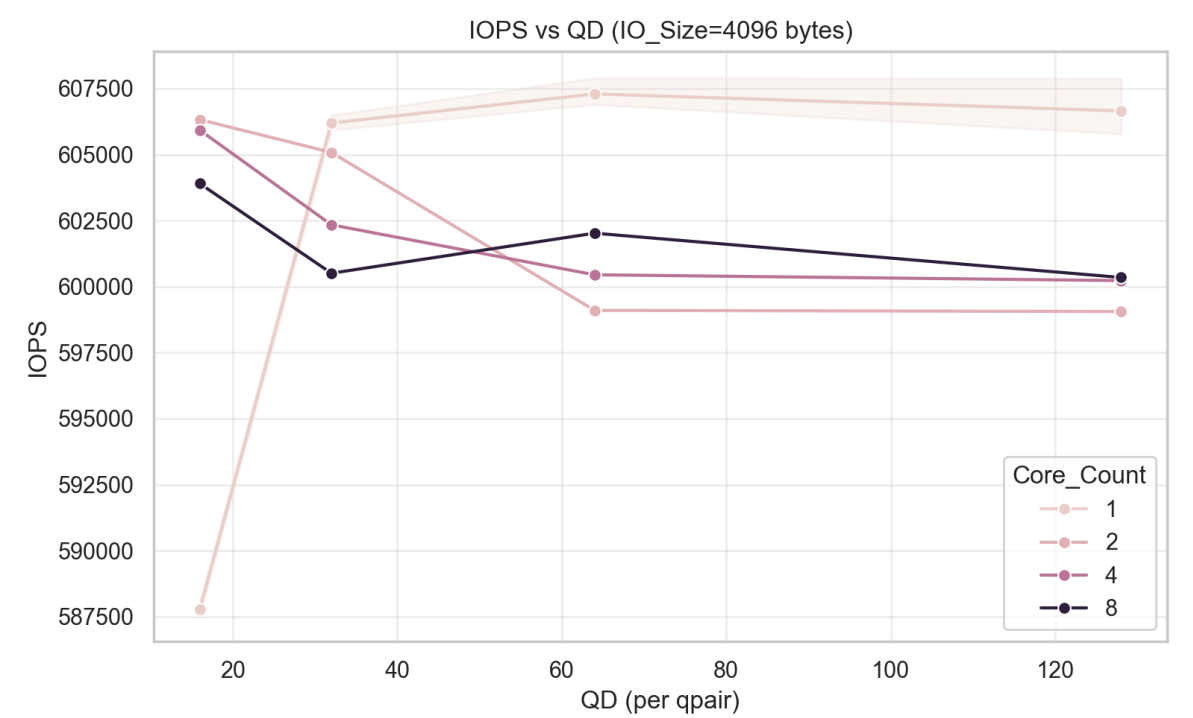
1 CPU core, 5.3 GHz; PCIe4 x4

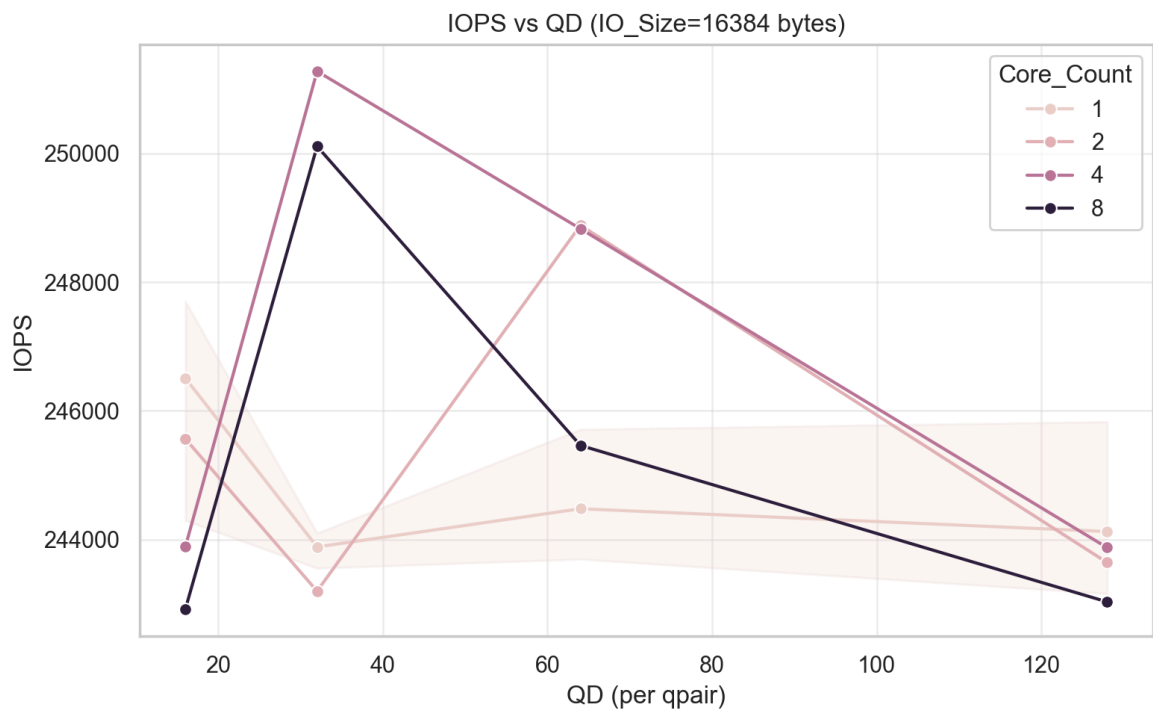
Iops Vs Qd



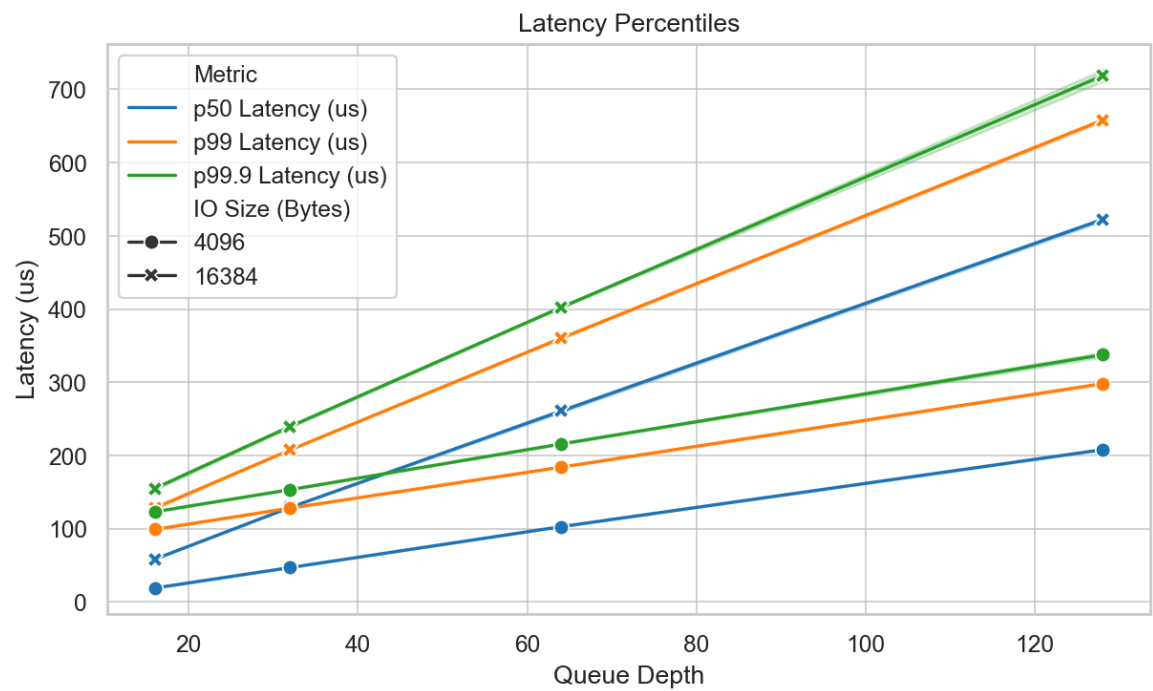
Insight: Saturation point identification.

Multi-core IOPS



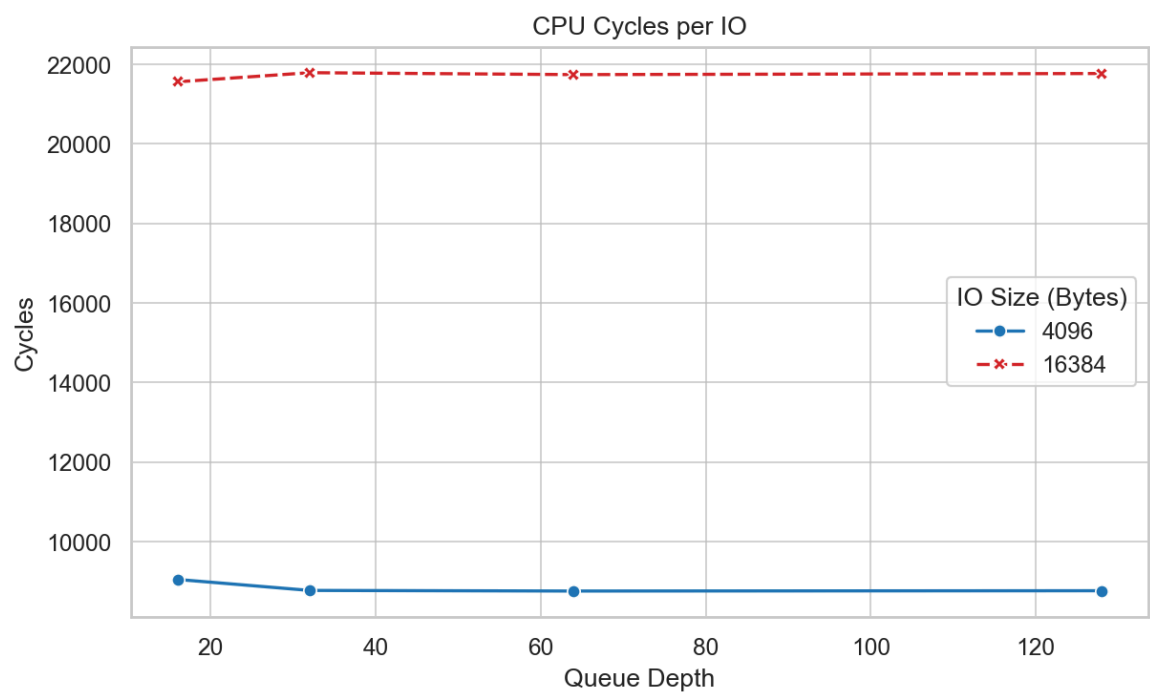


Latency Vs Qd



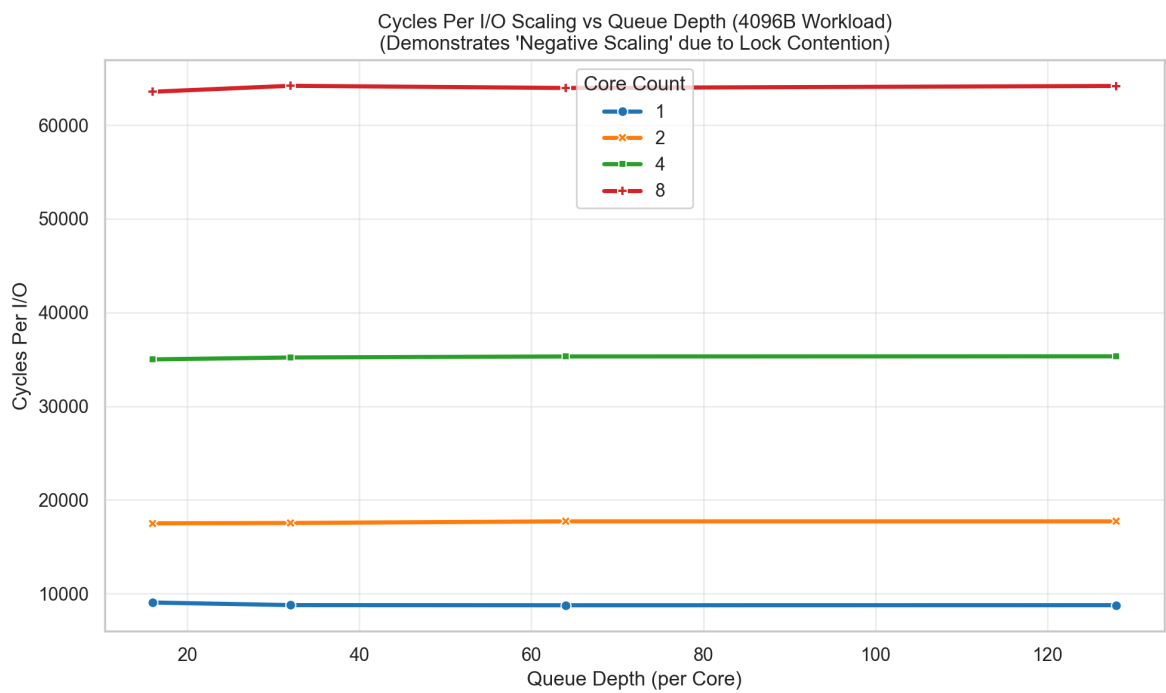
Insight: Tail latency characteristics.

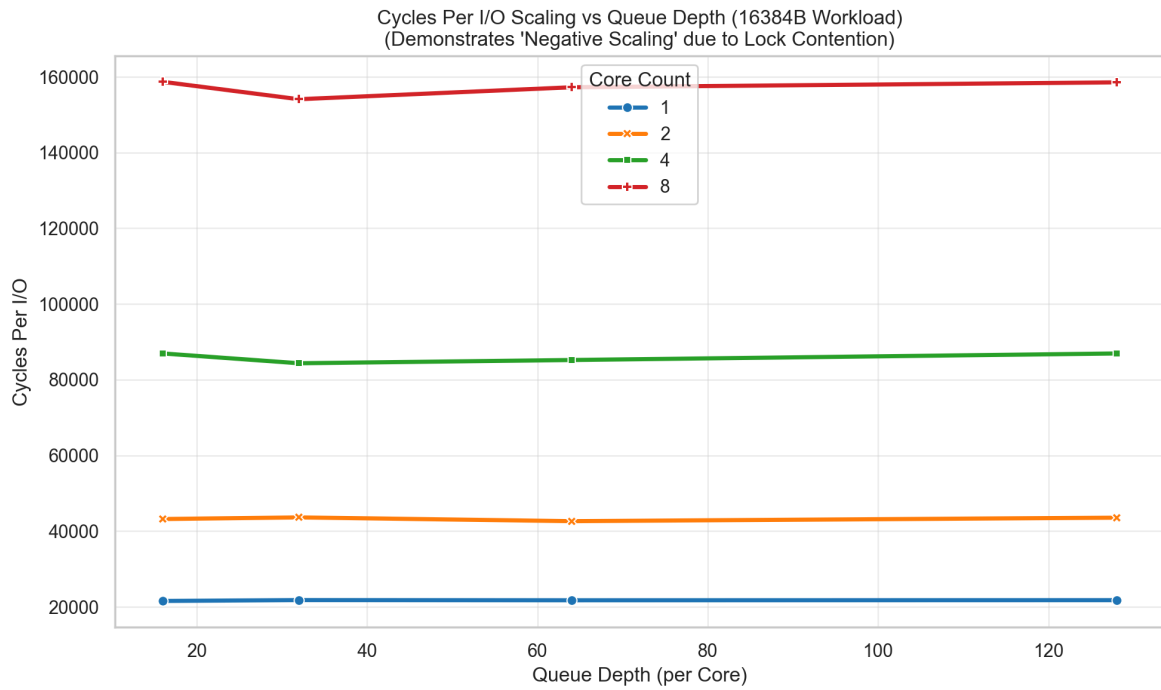
Cycles Per Io



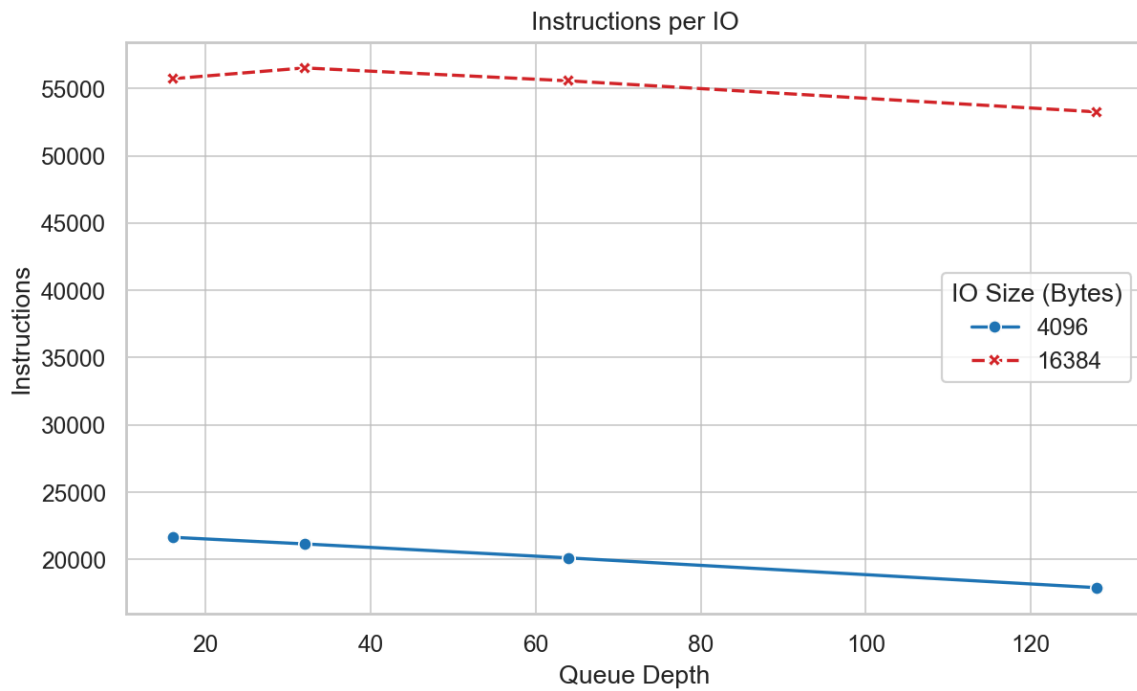
Insight: Total CPU cost per IO operation. Lower is better. (total time core active)/(total IOs)

Multi-core cycles per I/O



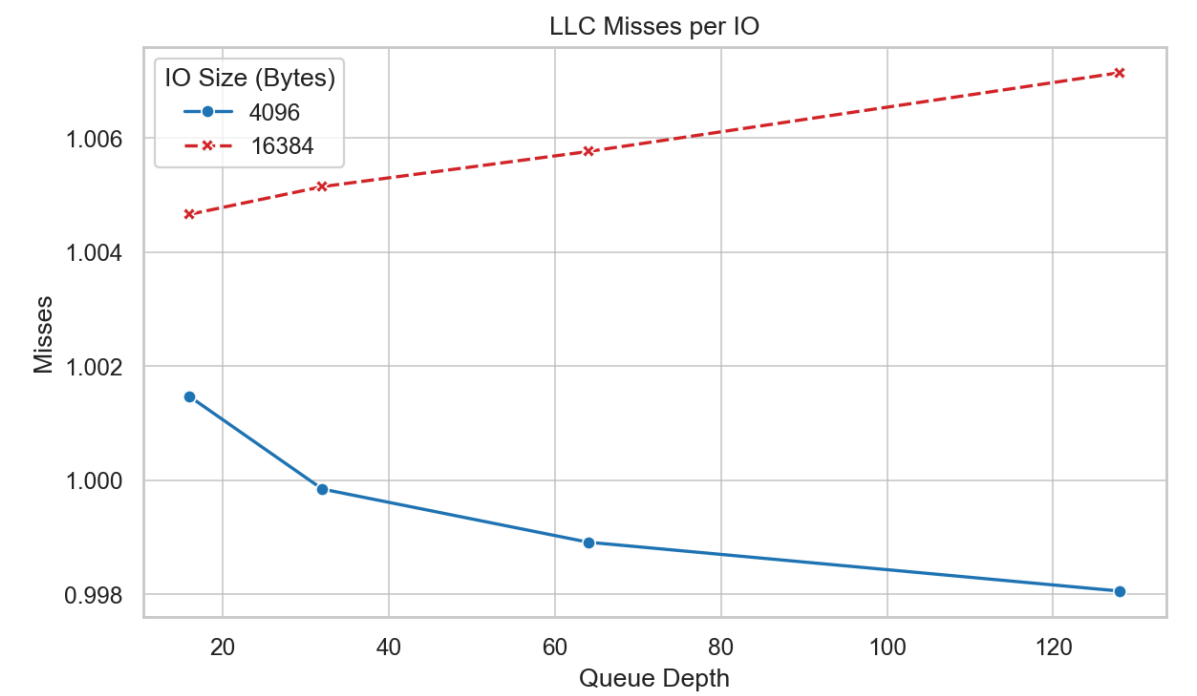


Instr Per Io



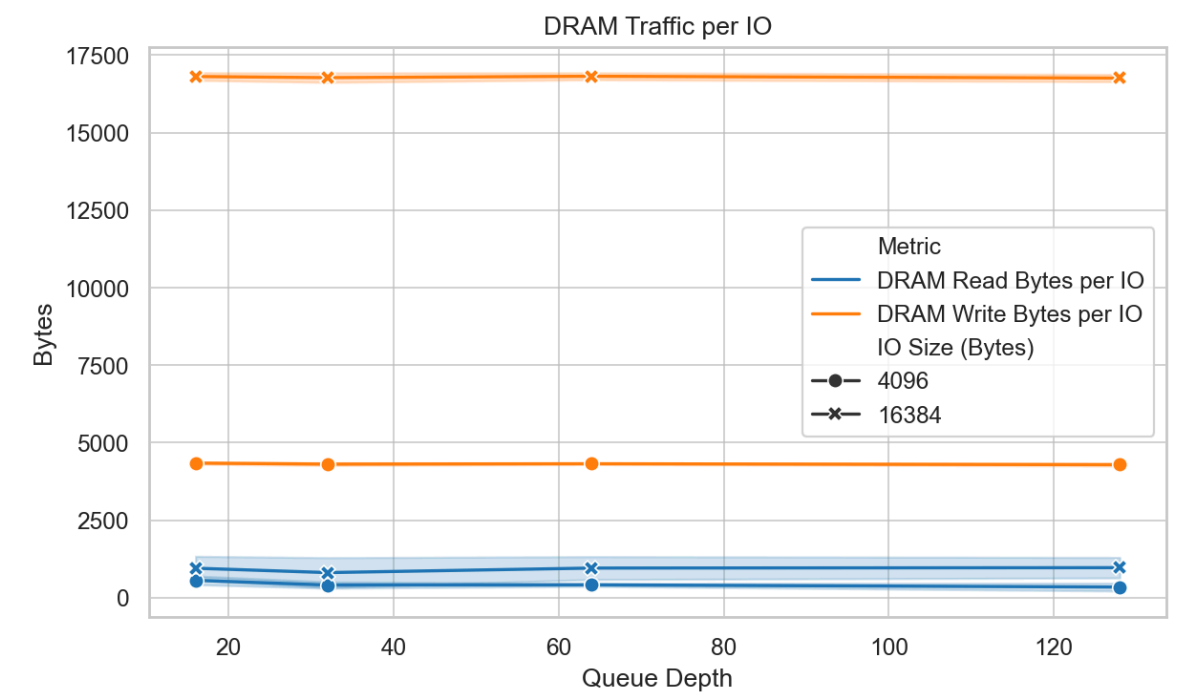
Insight: Algorithmic complexity per IO. High values at low QD indicate inefficient polling loops.

Llc Misses Per Io



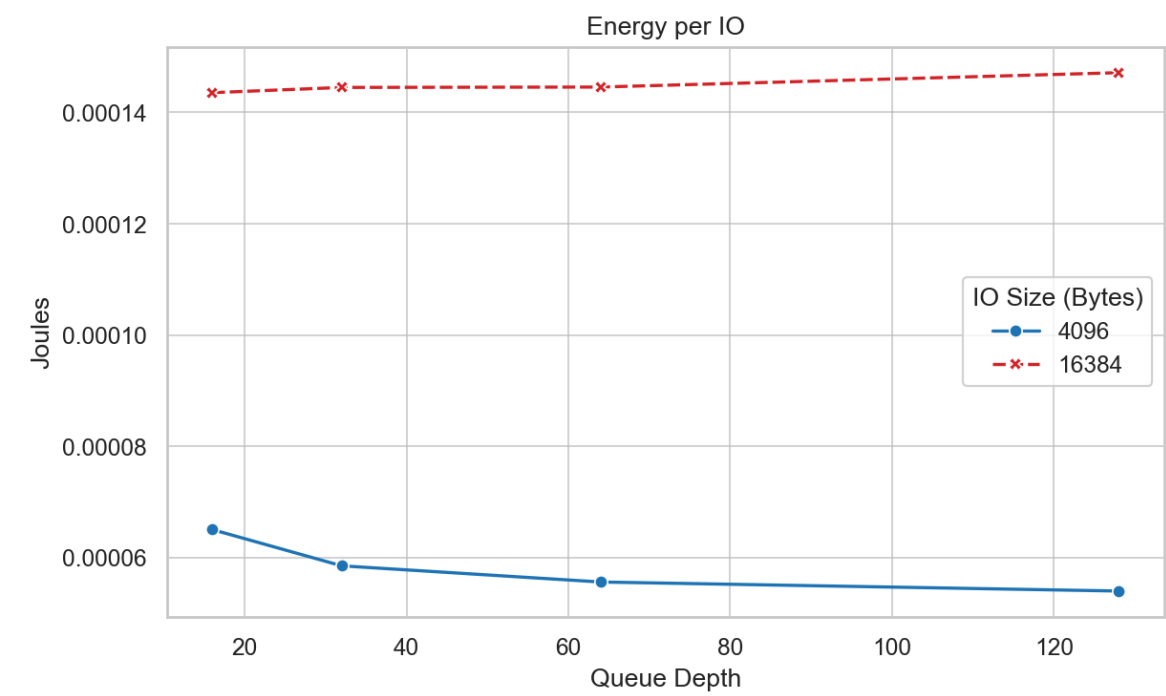
Insight: Last-Level Cache pressure. Indicates how much metadata/payload is hitting DRAM.

Dram Bw Per Io



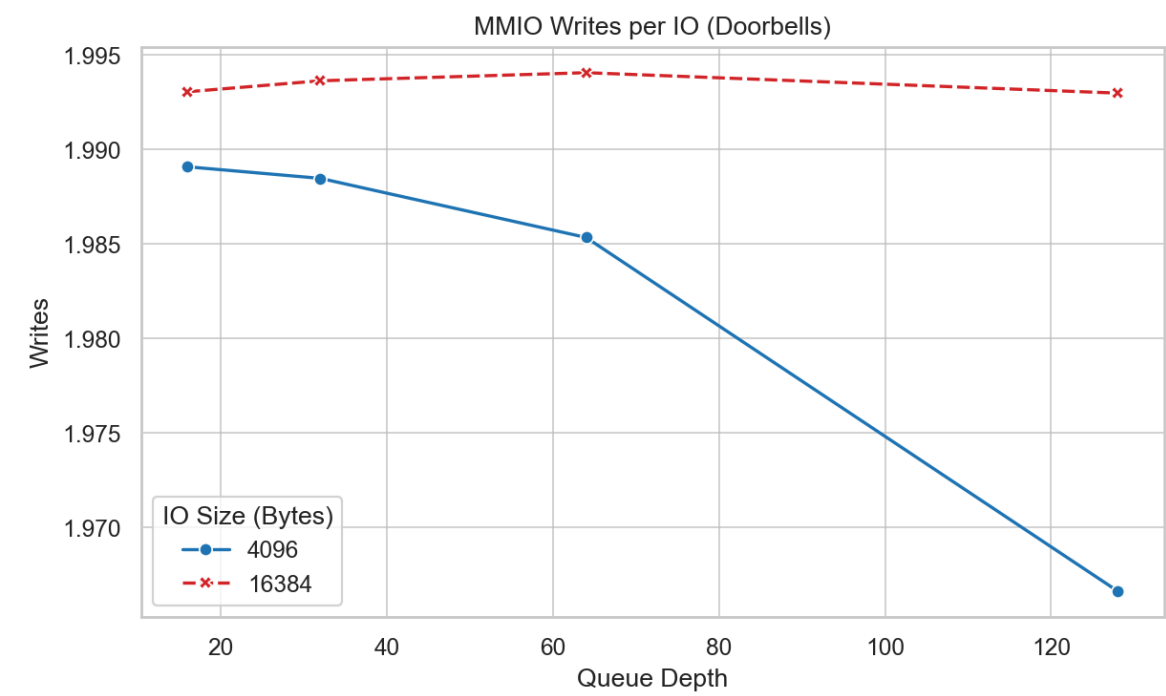
Insight: Data movement overhead. Should ideally match payload size, but often exceeds it due to descriptors.

Energy Per Io



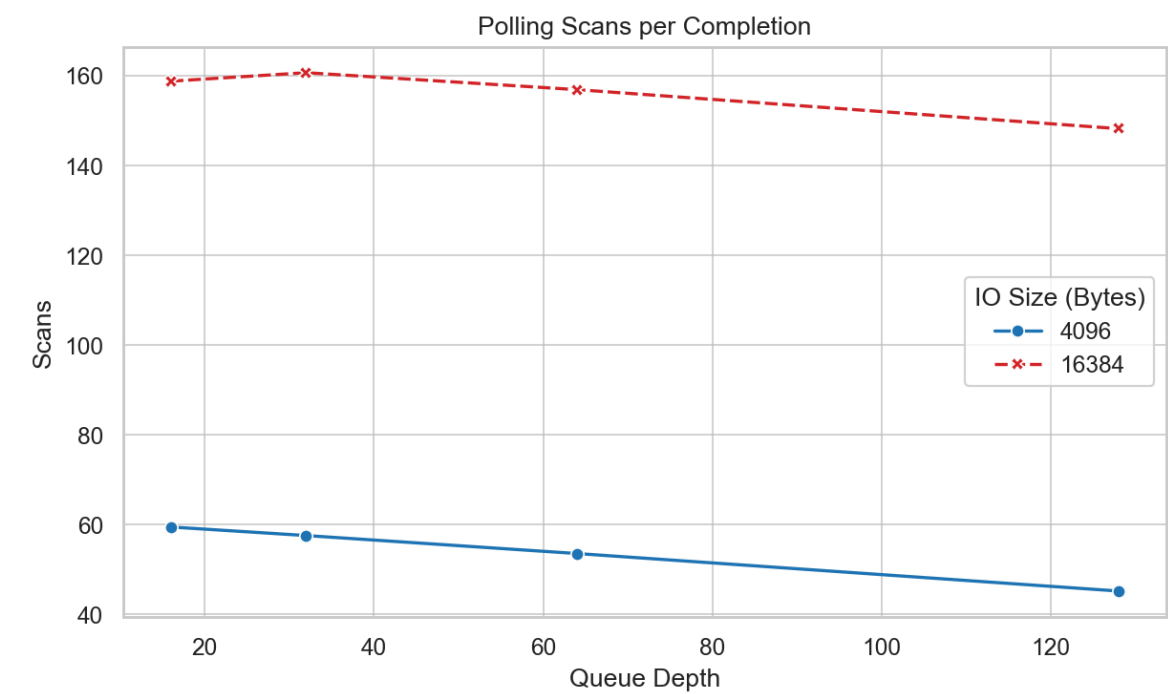
Insight: System energy cost per operation. (Note: May include idle power).

Mmio Per Io



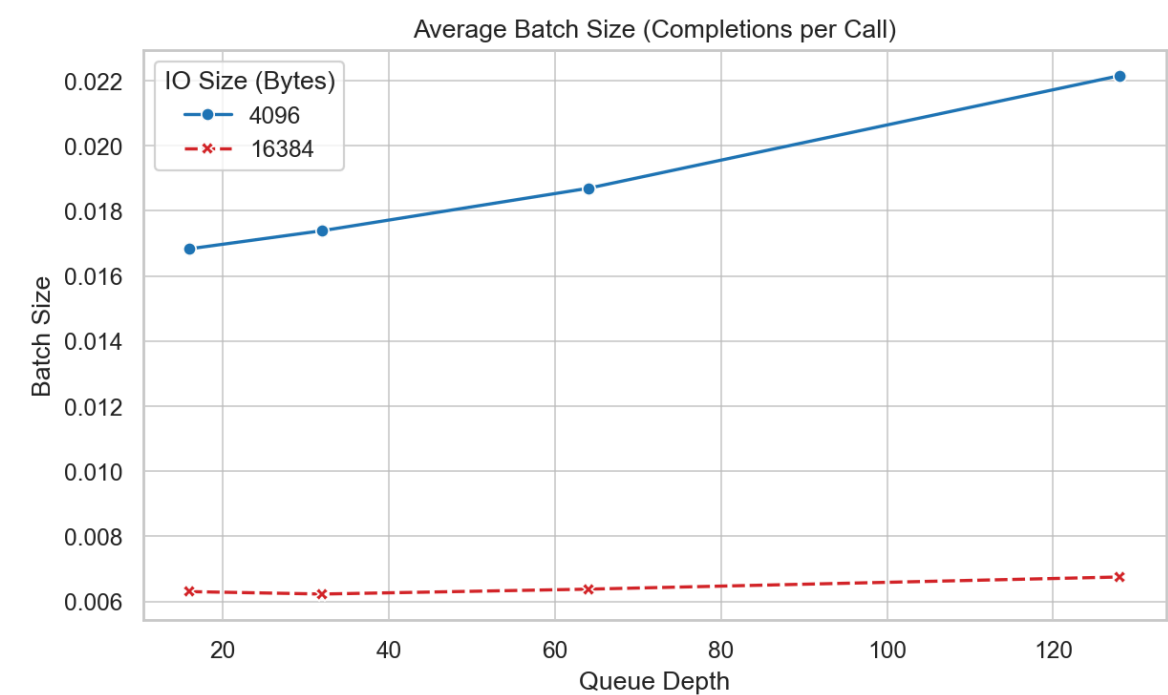
Insight: Hardware signaling cost. Values near 2.0 (SQ+CQ) indicate no batching; values < 1.0 indicate batching.

Scans Per Completion



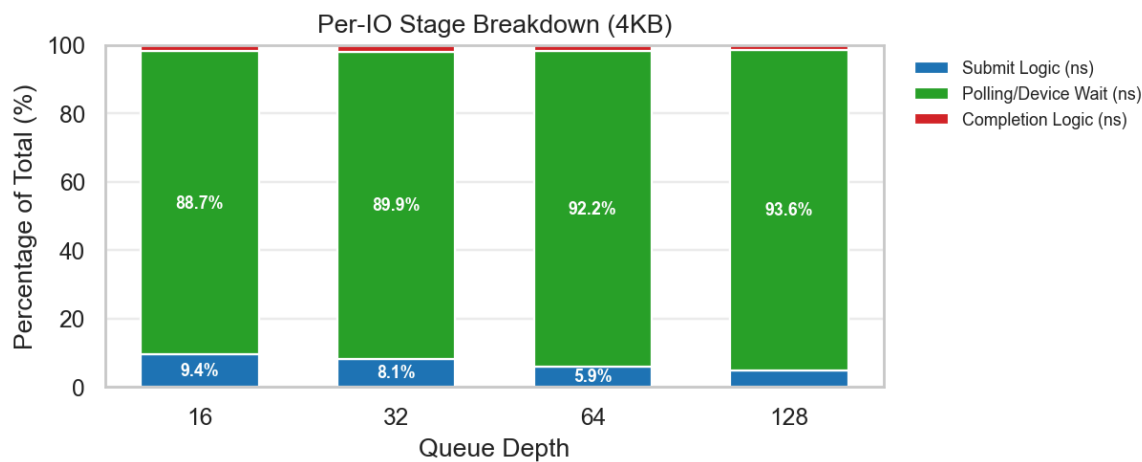
Insight: Wasted work metric: How many times the CPU checked an empty ring. High QD means more completions in Queue.

Batch Size



Insight: Average number of completions processed when work is actually found.

Stage Breakdown 4K



Insight: Submit logic:

Allocation: Getting the next free slot in the SQ.

Construction: Constructing and copying the 64B SQE into the SQ.

Barrier: Executing sfence (Write Memory Barrier) so SQE is visible before ringing the doorbell.

MMIO write: SQ tail update. May flush in batch for high queue depth.

Polling/Device Wait: time between doorbell ring and completion observed for this IO.

Completion logic:

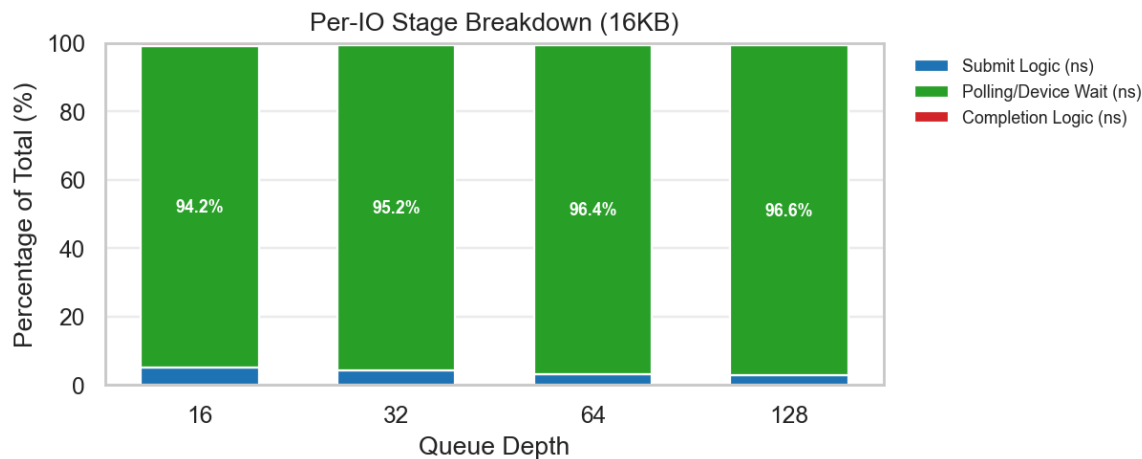
Detection: Reading the phase bit in the CQE.

D-Cache Invalidation (Implicit): CPU stalls while fetching the CQE line from DDIO/LLC.

Callback: Executing the user's completion callback.

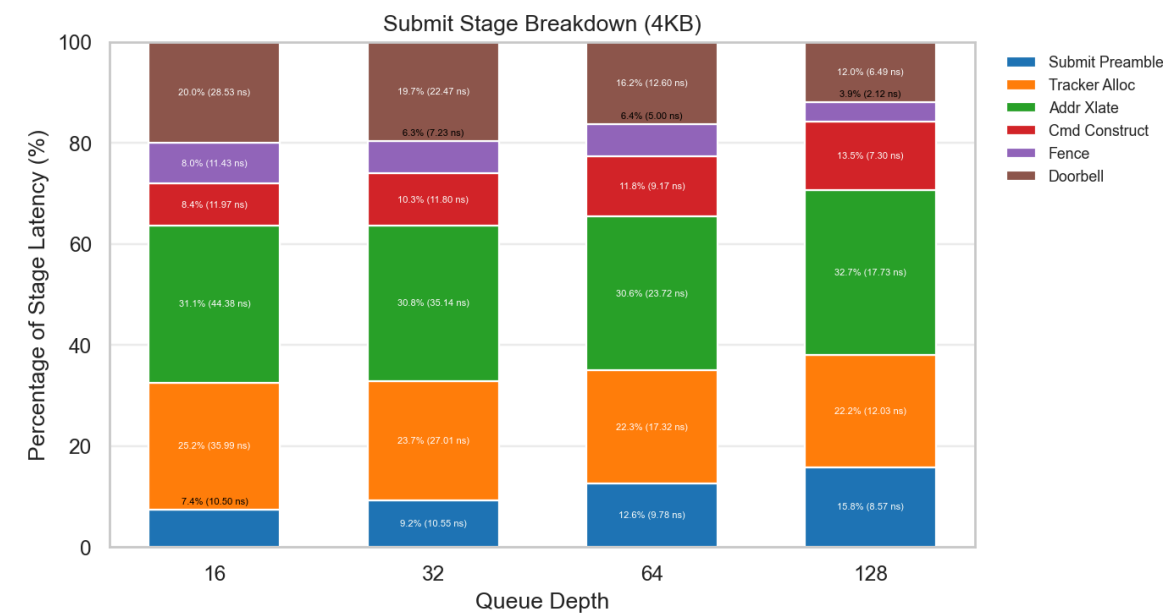
CQ Doorbell: Writing CQ head doorbell (often batched).

Stage Breakdown 16K



Insight: Stacked breakdown of per-IO time (converted to cycles) for 16KB requests.

Submit Stage Breakdown 4K



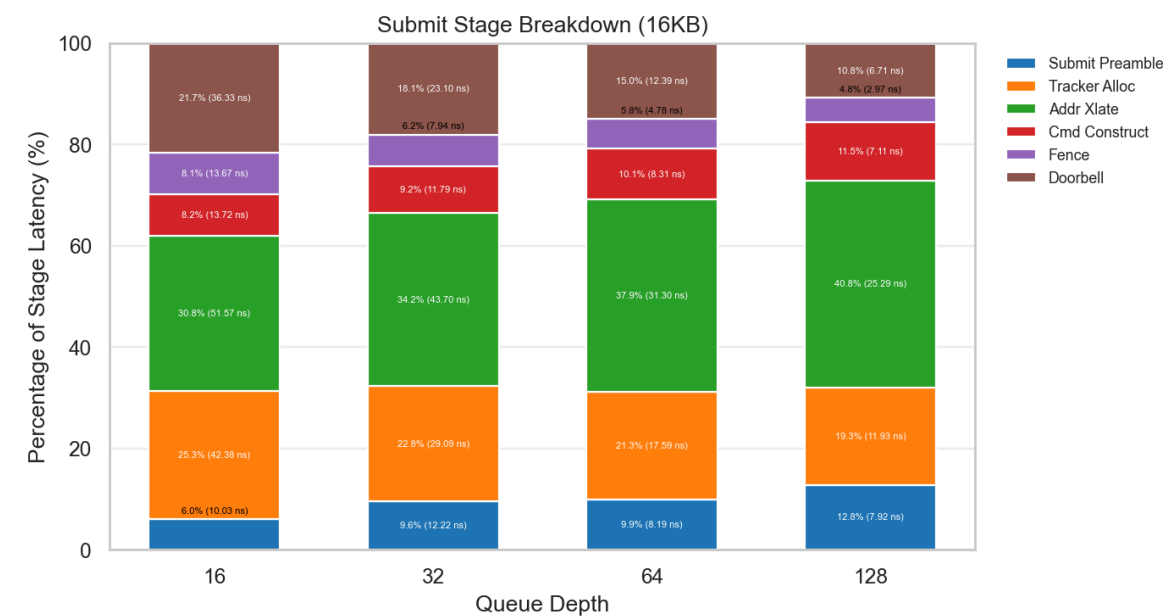
Insight: Submit stage percentages and per-step latency for 4KB.

Submit Preamble: driver verifies queue availability by reading the SQ tail and CQ head to ensure flow control credit exists.

Tracker Alloc: pop free nvme_tracker object to map user's request to specific CID. With low QD, CPU finishes the job quickly and then handle other jobs. The free list object may be evicted from L1 cache so next time allocating tracker, it needs to be fetched from maybe LLC.

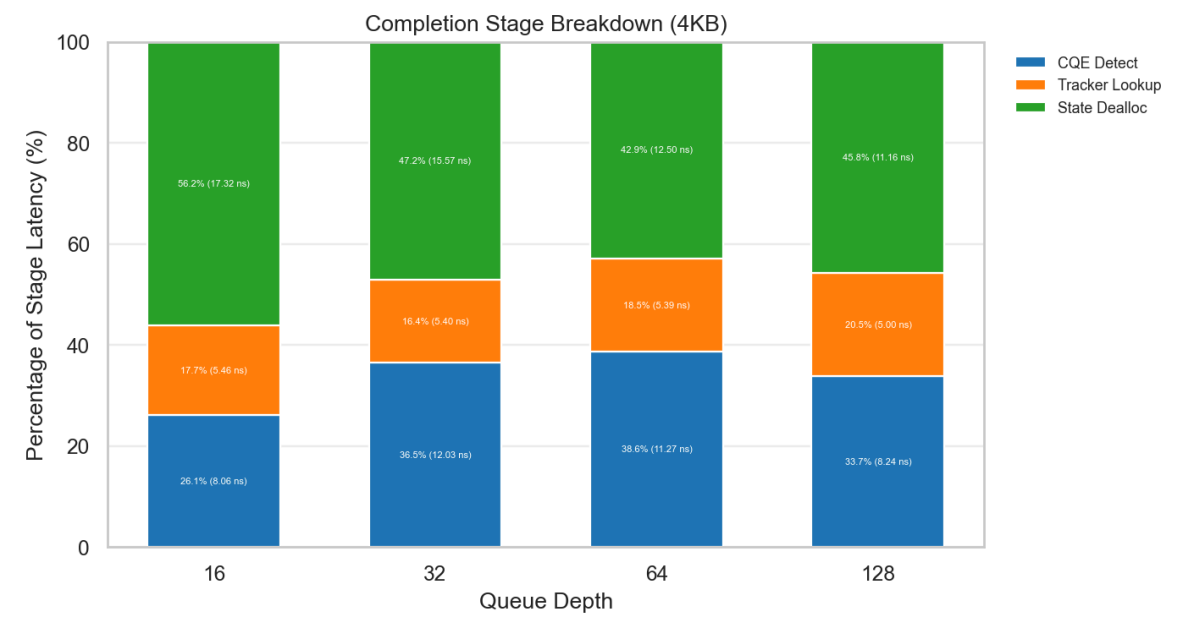
Addr Xlate: Address translation, also include calculating offsets and writing physical region page (PRP) list.

Submit Stage Breakdown 16K



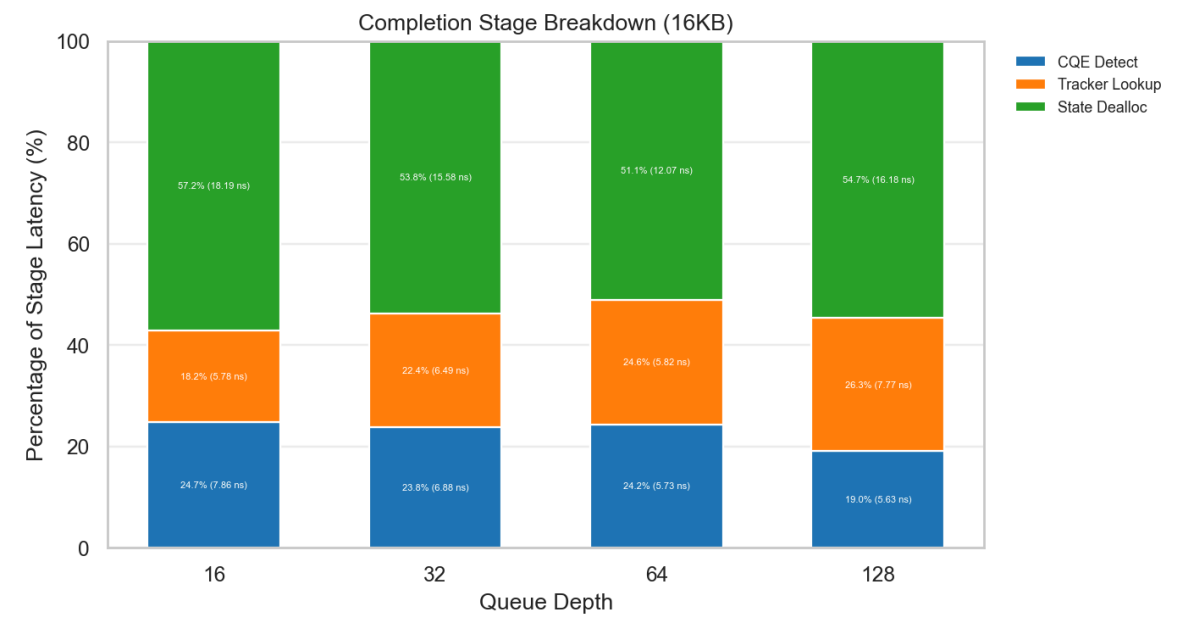
Insight: Submit stage percentages and per-step latency for 16KB.

Completion Stage Breakdown 4K



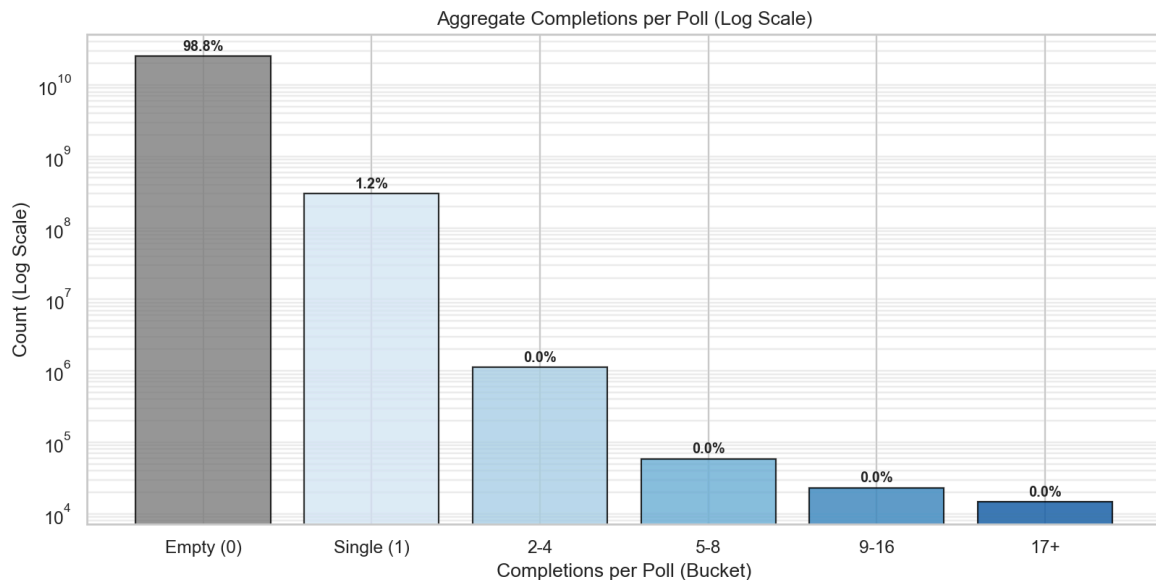
Insight: Completion stage percentages and per-step latency for 4KB. CQ doorbell is batched.

Completion Stage Breakdown 16K



Insight: Completion stage percentages and per-step latency for 16KB.

Completions Hist Log



Insight: Global distribution of polling outcomes.

How much DRAM traffic per I/O is metadata vs payload?

The Breakdown

- **4KB Workload:**
 - **Total DRAM Traffic:** ~4,630 Bytes per I/O.
 - **Payload:** 4,096 Bytes.
 - **Metadata Overhead: ~533 Bytes (13% Overhead).**
- **16KB Workload:**
 - **Total DRAM Traffic:** ~17,725 Bytes per I/O.
 - **Payload:** 16,384 Bytes.
 - **Metadata Overhead: ~1,340 Bytes (8.2% Overhead).**

Larger Payload PRPs Tax:

- Metadata **doubles** from 4KB (~533B) to 16KB (~1,340B).
- **Reason:** As discussed, 4KB fits in standard descriptors. 16KB requires constructing and fetching an extra **PRP List Page** from DRAM. This extra "pointer chasing" adds ~800 bytes of read/write traffic just to describe where the data lives.

Metadata is not negligible. It consumes **8-13%** of your system's memory bandwidth. An I/O Uncore that manages this state internally (SRAM) would not only save CPU cycles but also free up significant DRAM bandwidth for the application.

How do costs scale with qpairs, QD, and devices?

For QD increases, individual software components become much more efficient.

For 4KB Workloads (IOPS Intensive):

- **Submission Logic:** Drops by **50%** (160 ns → 80 ns).
 - *Reason:* Hotter caches and better branch prediction for the submission loop.
- **Polling Waste (`Scans_Per_Completion`):** Drops by **24%** (59 scans → 45 scans).
 - *Reason:* With more requests in flight, the probability of finding a completion on any given check increases.
- **Completion Logic:** Drops by **~20%** (32 ns → 26 ns).
 - *Reason:* Batching effects and instruction cache locality.

For 16KB Workloads (Bandwidth Intensive):

- **Submission Logic:** Drops by **45%** (200 ns → 109 ns).
- **Polling Waste:** Drops slightly by **6.7%**.
 - *Reason:* The CPU is waiting much longer for the SSD to transfer the larger payload, so the "fishing" is harder.

Total Cycles per I/O barely moves, saved cycles spend on polling:

- **4KB:** 9,044 cycles → 8,761 cycles (**-3%**).
- **16KB:** 21,563 cycles → 21,772 cycles (**+1%**).

How many cores are required as aggregate IOPS increases?

For single QP, allocating more cores (e.g., 4 or 8) does not increase performance. Instead, it forces the cores to fight for the single lock, causing them to burn 4x to 8x more CPU cycles just to achieve the same result as a single core.